

Interface e Gerenciamento de Serviços no Android

Para alcançar os objetivos definidos para essa atividade, foi implementado um aplicativo simples que gera chaves de acesso aleatórias ao pressionar um botão na tela. Os tópicos a seguir descreverão mais sobre o funcionamento do aplicativo e seu diagrama de solução.

Objetivo

- Implementar a estrutura e o funcionamento do Binder no Android;
- Implementar uma interface AIDL em um serviço Android integrada ao Binder.

Repositório

Link do Repositório: [gitHub](#)

Link da Aplicação: [AIDL_Interface](#)



Nota Este link é da branch do repositório onde se encontra o código-fonte da aplicação. Após a avaliação da atividade, a branch será megeada na devel.

Sumário

1. [Ambiente de desenvolvimento](#)
 - 1.1. [Sistema Operacional](#)
 - 1.2. [Java\(JDK\)](#)
 - 1.3. [Android Studio](#)
 - 1.4. [Git](#)
 - 1.5. [Emulador Android](#)
2. [Resultado da Atividade](#)
 - 2.1. [Estrutura de pastas do repositório](#)
 - 2.2. [Implementação](#)
 - 2.3. [Resultados](#)

1. Ambiente de Desenvolvimento

1.1. Sistema Operacional

```
# O comando lsb_release imprime certas informações de LSB (Linux Standard
Base) e distribuição.
$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 22.04.5 LTS
Release: 22.04
Codename: jammy
```

1.2. Java(JDK)

```
# Retorna informações do Java caso esteja instalado
$ java --version
```

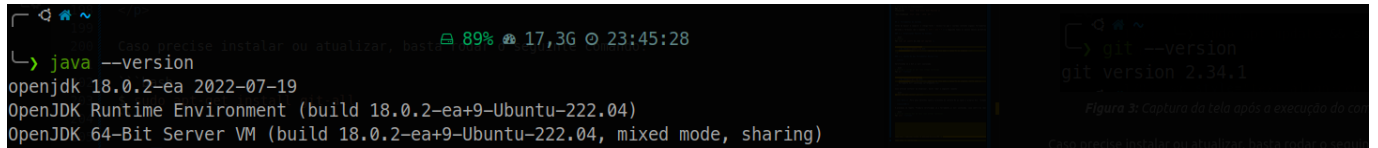


Figura 1: Captura da tela após a execução do comando, ferramenta instalada corretamente

1.3. Android Studio

```
# Informações gerais do Android Studio

Android Studio Meerkat | 2024.3.1
Build #AI-243.22562.218.2431.13114758, built on February 24, 2025
Runtime version: 21.0.5+-12932927-b750.29 amd64
VM: OpenJDK 64-Bit Server VM by JetBrains s.r.o.
Toolkit: sun.awt.X11.XToolkit
Linux 6.8.0-52-generic
GC: G1 Young Generation, G1 Concurrent GC, G1 Old Generation
Memory: 2968M
Cores: 12
Registry:
  ide.experimental.ui=true
  i18n.locale=
Current Desktop: ubuntu:GNOME
```

1.4. Git

```
# Retorna a versão do Git caso esteja instalado
$ git --version
```

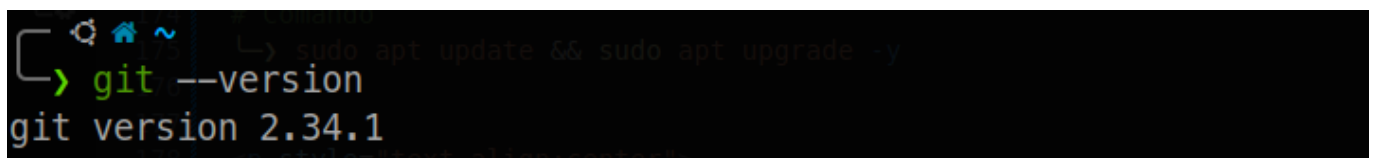


Figura 2: Captura da tela após a execução do comando, neste caso ferramenta está instalada corretamente

1.5. Emulador Android

O processo de criação de um dispositivo virtual (AVD) já foi realizado em atividades anteriores. Para esse projeto, um novo dispositivo foi criado apenas para fins de prática.

Etapas da criação e configuração do dispositivo virtual:

- Abrindo o gerenciador de dispositivos virtuais.

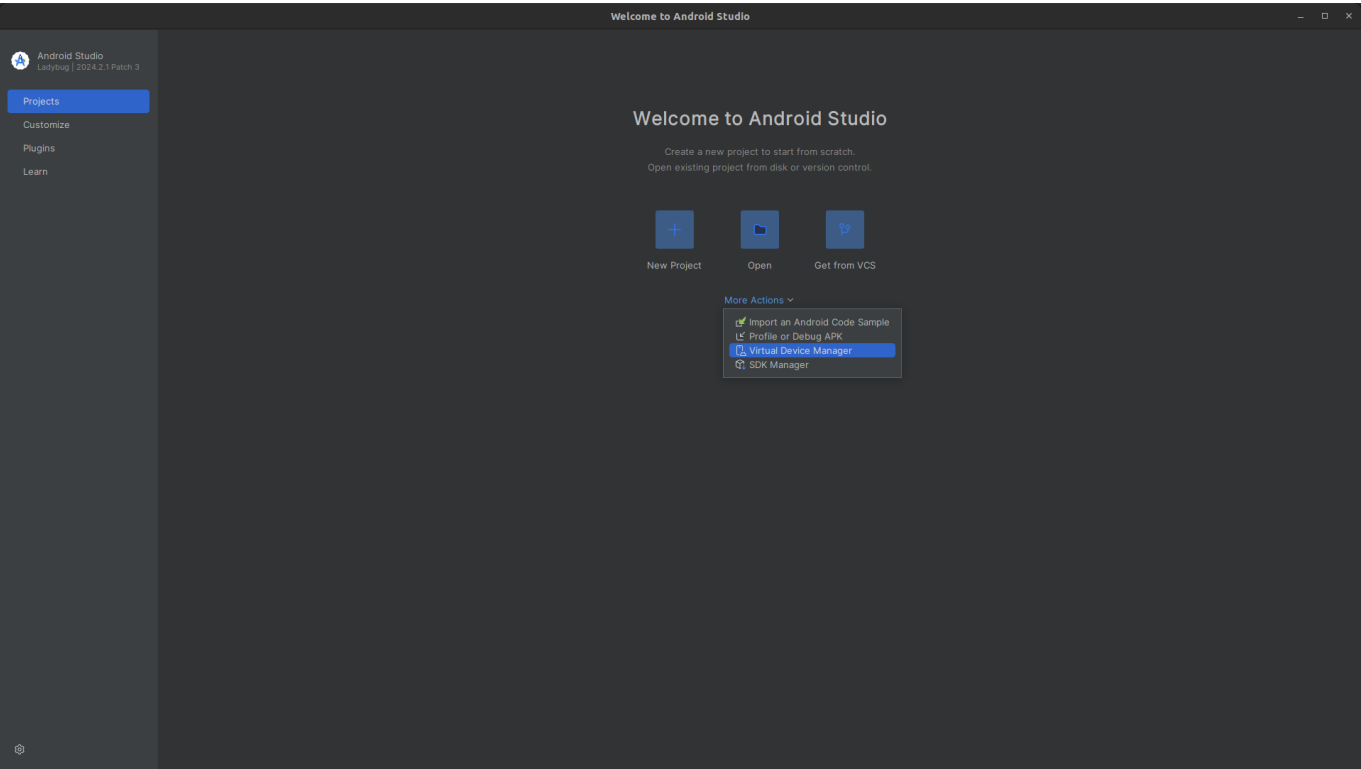


Figura 3: Configuração: Virtual Devices Manager

- Escolhendo a categoria e o modelo do dispositivo a ser criado.

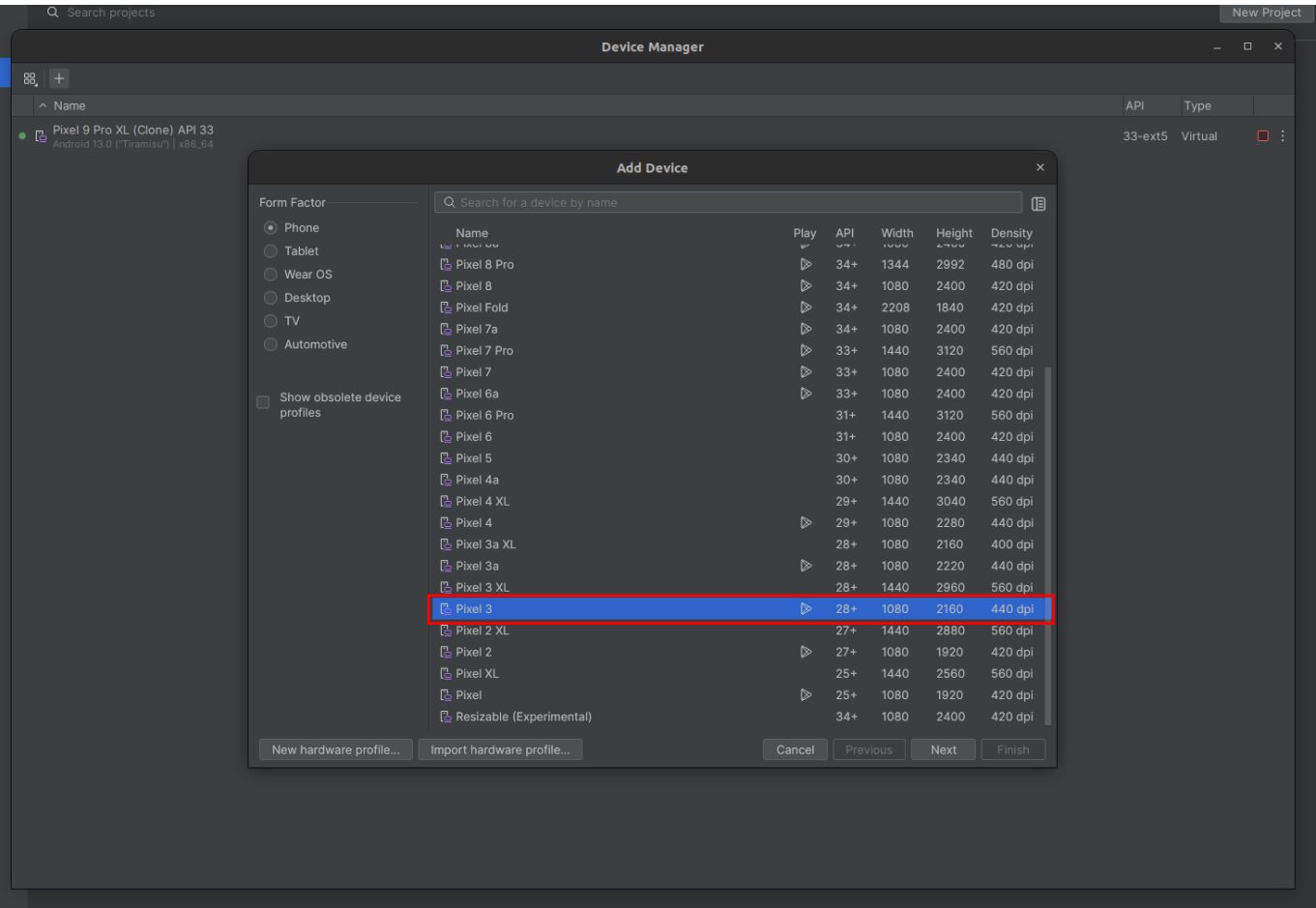


Figura 4: Seleção do Hardware

- Alteração do tamanho da RAM e da quantidade de núcleos do processador.

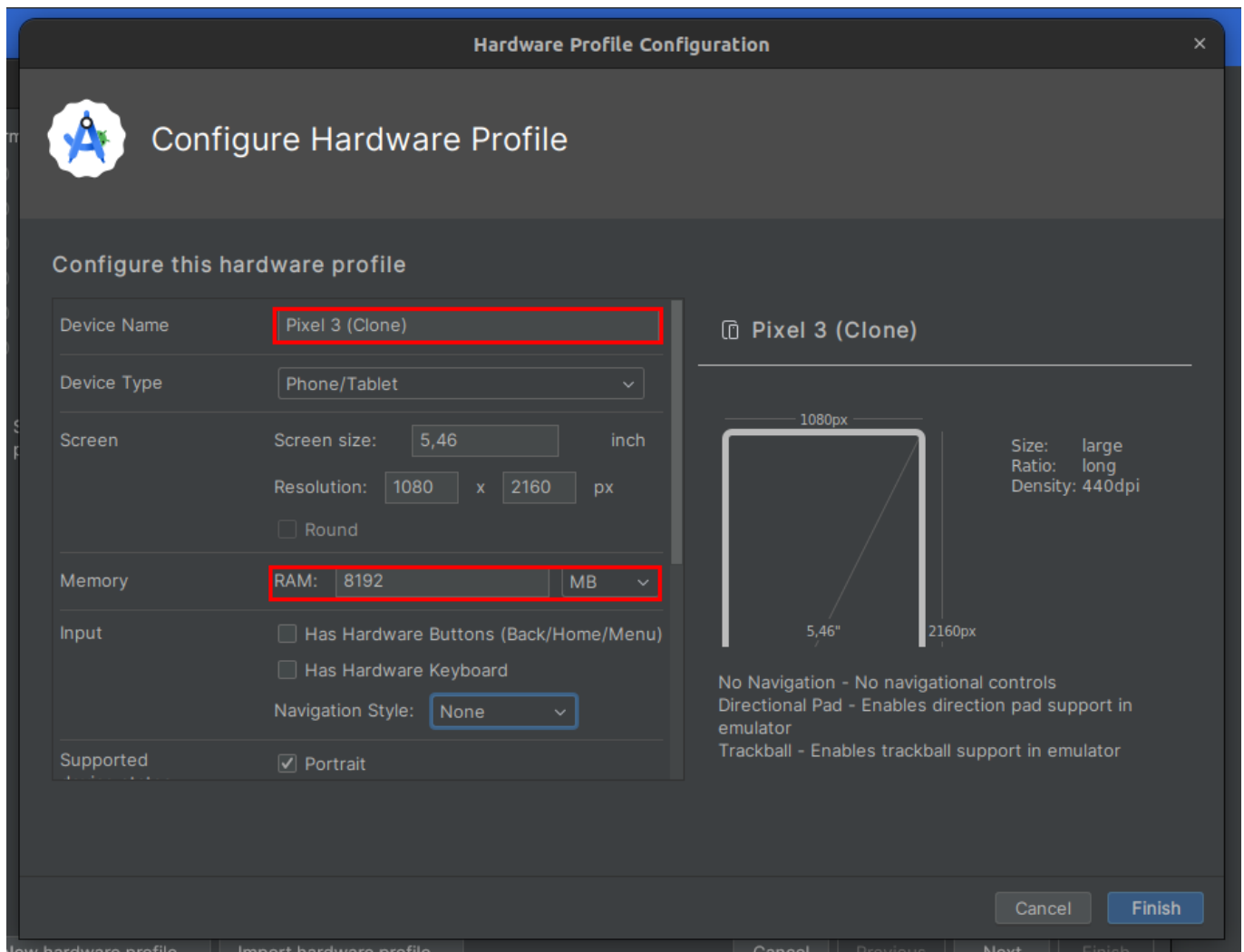


Figura 5: Ajuste da capacidade do hardware



Nota Prefiro clonar o dispositivo e alterar as configurações para o que melhor me interessa.

- Download da imagem do sistema.

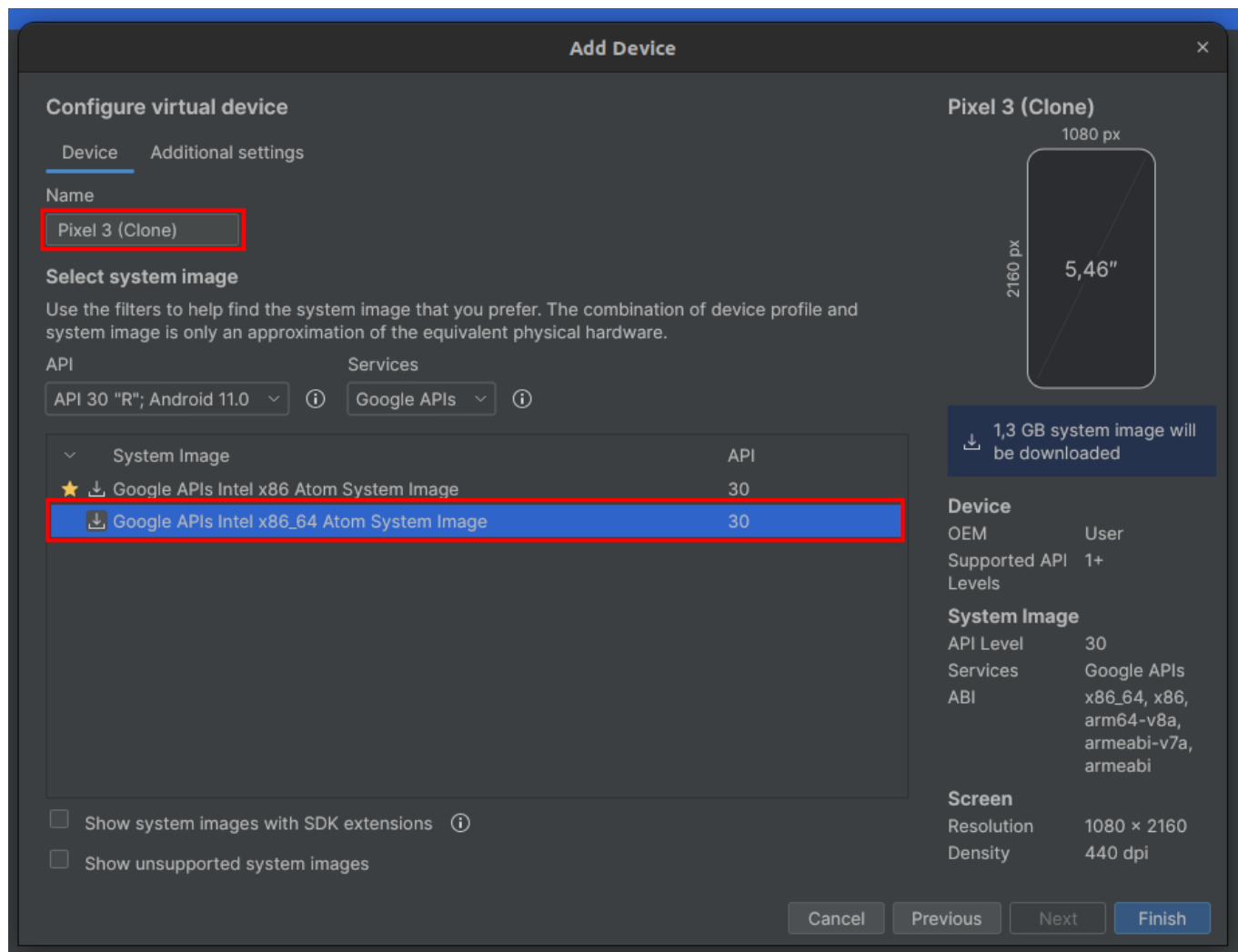


Figura 6: Seleção e download da imagem do sistema

- Criando dispositivo.

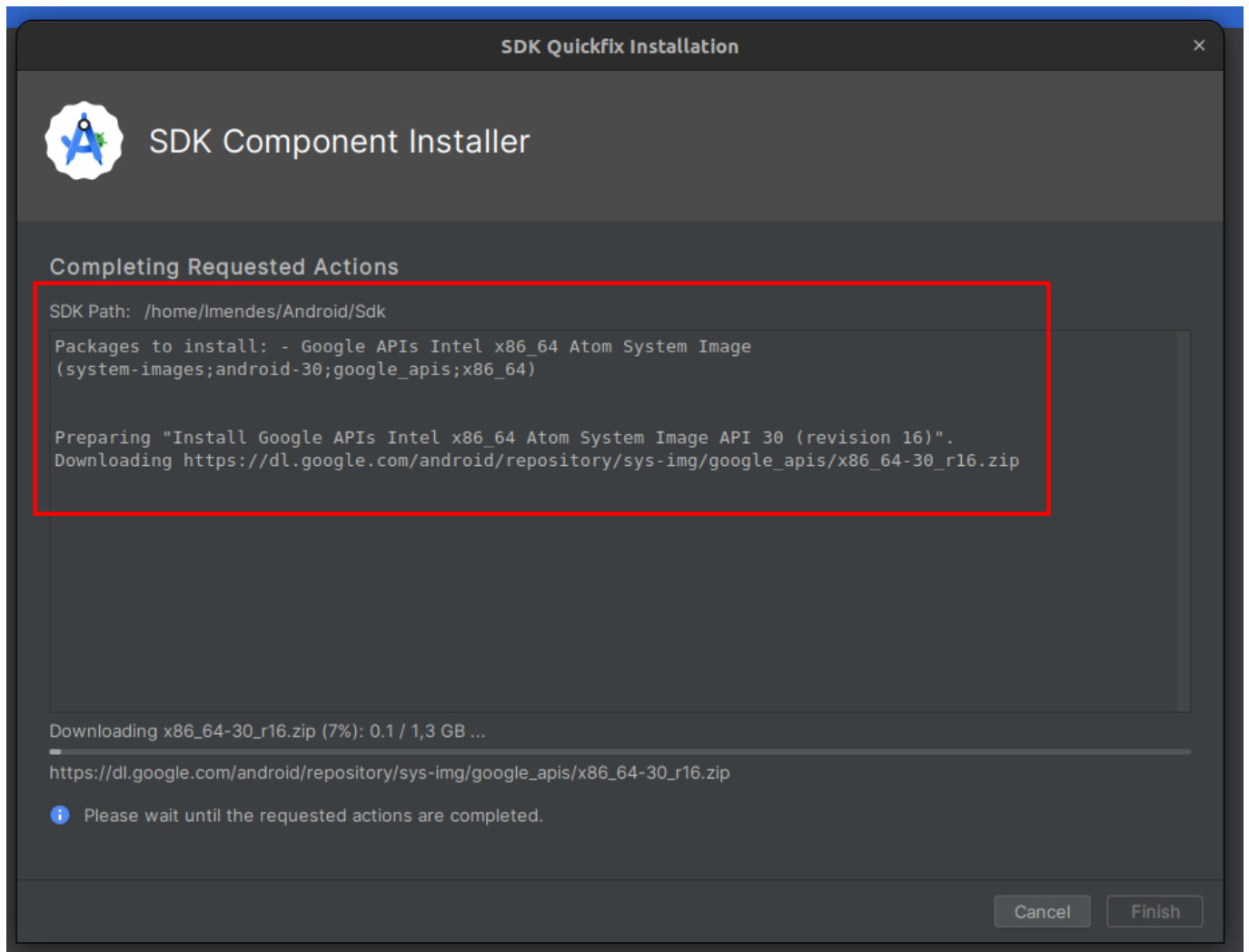


Figura 7: Instalando Componente

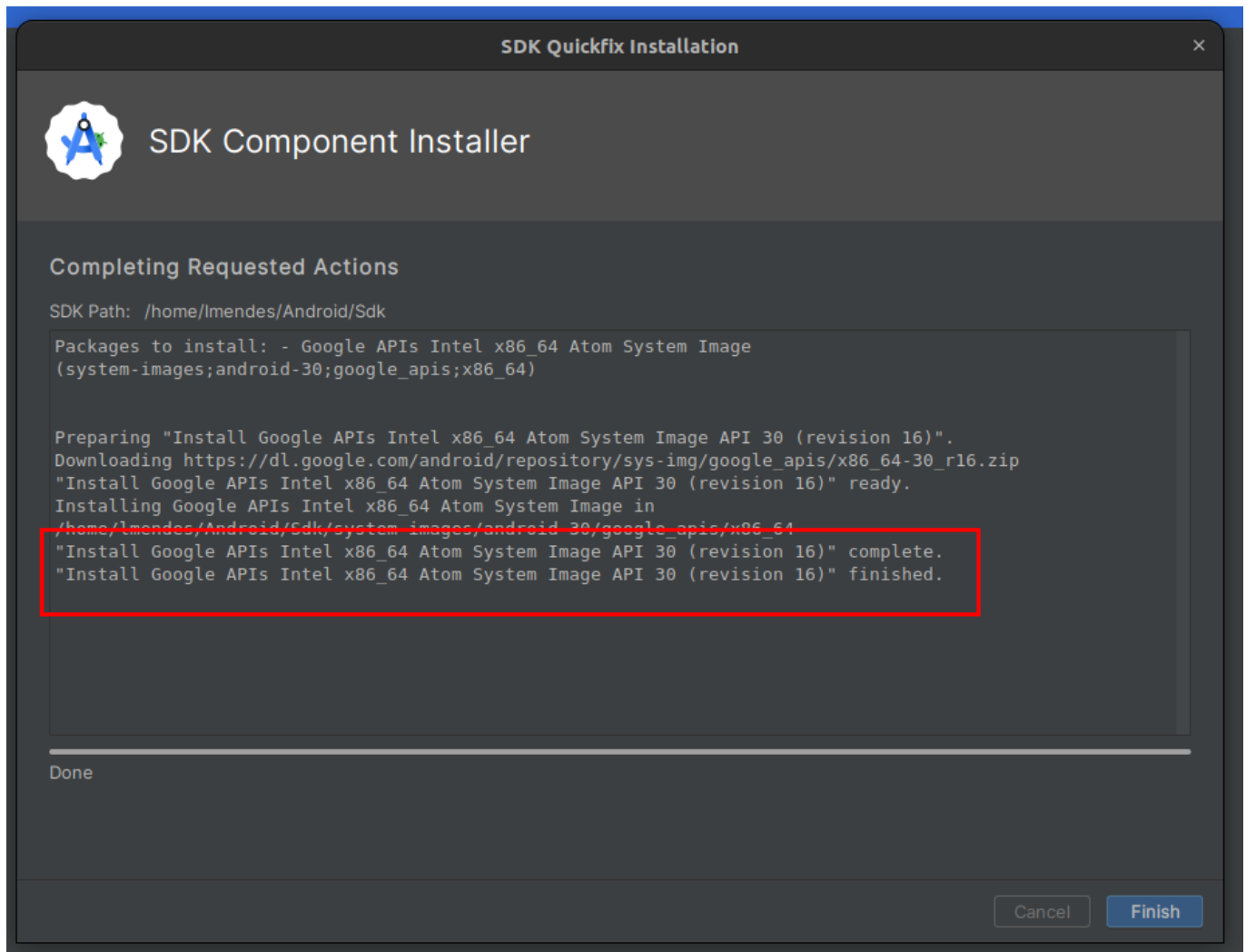


Figura 8: Dispositivo criado

2. Resultado da Atividade

2.1 Estrutura de pastas do repositório

```

.
├── aosp
│   ├── kernel # Arquivos de configuração e patch do kernel do Android
│   └── src # Aplicativos desenvolvidos durante o curso
│       └── apps # Aplicativos desenvolvidos até o momento
│           └── AIDL_Interface # Projeto Hands on UA1 e UA2 da disciplina:
Interface e Gerenciamento de Serviços no Android
├── app
├── build
├── build.gradle.kts
├── .gitignore
├── .gradle
├── gradle
├── gradle.properties
├── gradlew
├── gradlew.bat
├── .idea
└── local.properties
  
```

```

├── settings.gradle.kts
├── audio_equalizer
├── docs
│   ├── reports # Todos os relatórios no formato PDF
├── readme_files # Todos os relatórios escritos em markdown
│   ├── AIDL_dc3_u1_u2.md
│   ├── android_application_dc2_p1.md
│   ├── aosp_customization.md
│   ├── environmental_preparation.md
│   └── imgs
├── README.md # LEIA-ME principal do repositório
├── .vscode
└── settings.json

```

O Caminho do projeto desenvolvido conforme a estrutura de pastas acima é

/aosp/src/apps/AIDL_Interface. Todos os arquivos do projeto do **android studio** estão dentro desta pasta.

2.2 Implementação

Interface AIDL implementa, possui apenas o método `int getRandomKey()` conforme figura abaixo.

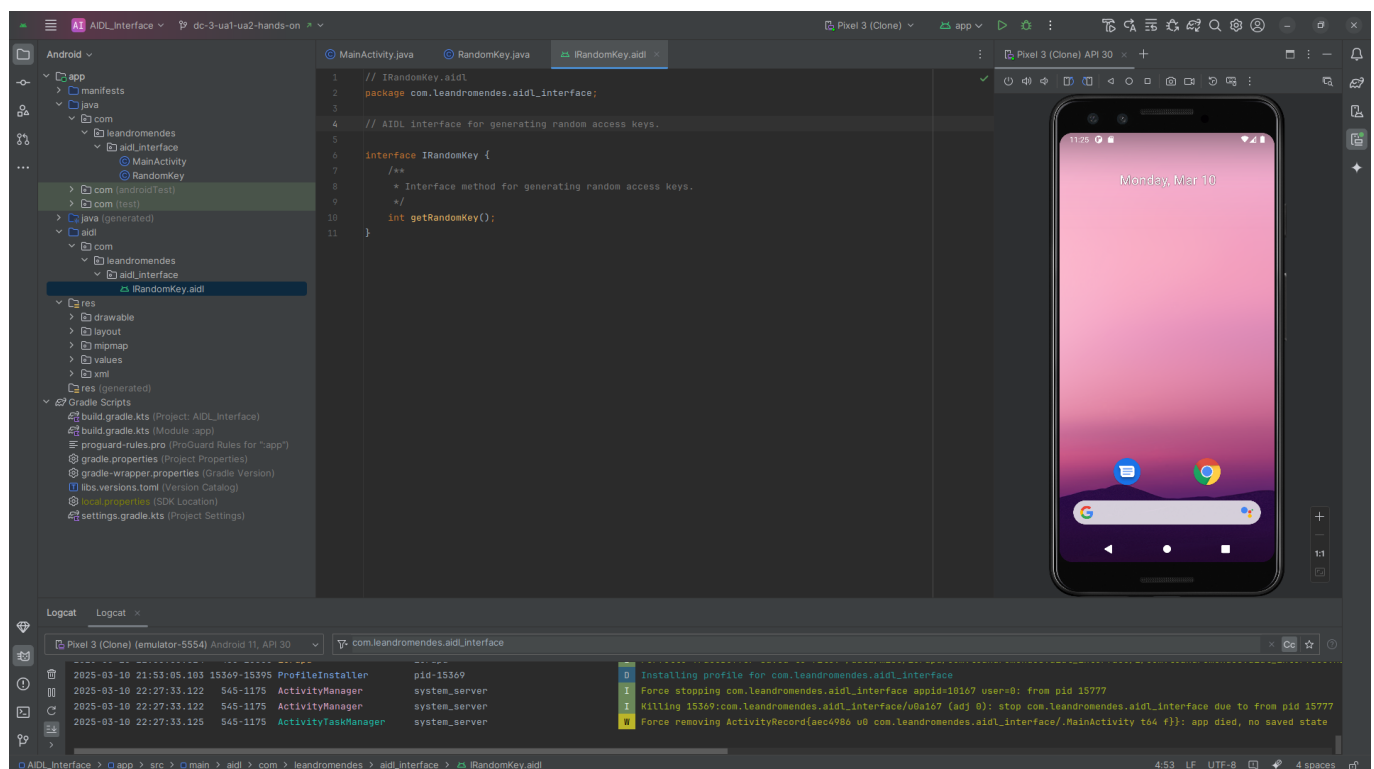


Figura 9: Interface AIDL

A classe `RandomKey.class` implementa o método definido na interface conforme a imagem abaixo. Essa classe estende o `services` responsável pela execução do método.

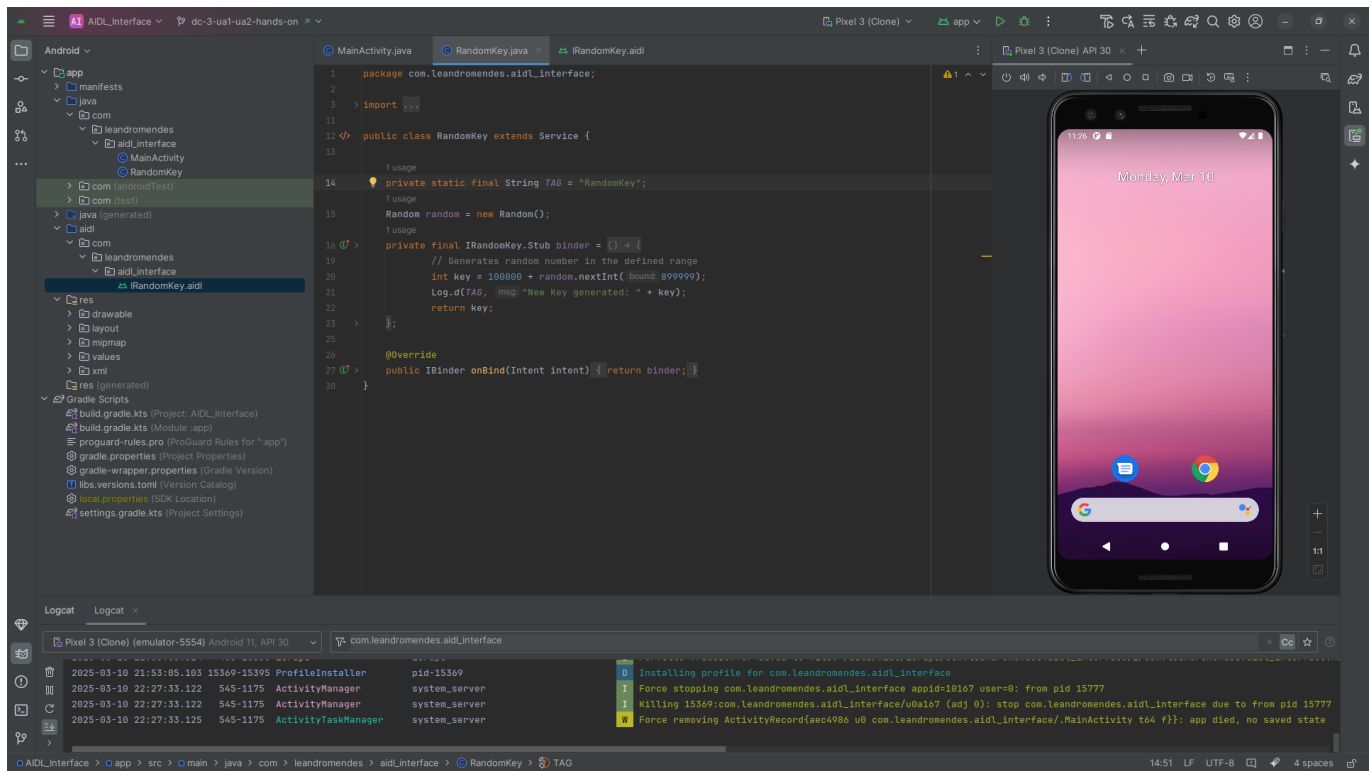


Figura 10: Service

Quando a classe do serviço é criada usando o Android Studio, algumas configurações adicionais são criadas no arquivo Manifest do android.

```
<service
    android:name=".RandomKey"
    android:exported="true"
    android:permission="android.permission.BIND_REMOTE_SERVICE" >
    <intent-filter>
        <action android:name="com.leandromendes.aidl_interface.IRandomKey"
    />
    </intent-filter>
</service>
```

A permissão e o intent-filter foram adicionados a configuração criada.

O processo de conexão do binder ocorre no `onCreate` chamado na `MainActivity.class` conforme o trecho abaixo.

```
// Create an intent for a specific component
// Create intent
Intent intent = new Intent();
// Define the component
intent.setComponent(
    new ComponentName(
        "com.leandromendes.aidl_interface",
        "com.leandromendes.aidl_interface.RandomKey"));
```

```
// Connect the binder to the service
bindService(intent, serviceConnection, Context.BIND_AUTO_CREATE);
```

Toda vez que o botão na interface gráfica é pressionado o serviço é acessado e retorna um número aleatório representando uma chave de acesso aleatória conforme o trecho abaixo.

```
// Listens when the button is pressed
generate.setOnClickListener(v -> {
    keyDisplay.setText(format("%d", generateKey()));
});

/*
 * Returns a random key generated by the service if it is connected.
 * Otherwise, it returns 0
 * */
private int generateKey() {
    if(isConnected){
        try {
            // Communicates with the service and returns the random key
            return randomKeyService.getRandomKey();
        }
        catch (RemoteException e) {
            e.printStackTrace();
        }
    }
    Log.d(TAG, "The service is not yet available!");
    return 0;
}
```

Algumas configurações foram necessárias para a correta execução do aplicativo, a primeira foi habilitar a flag `buildFeatures.aidl = true` no arquivo `build.gradle.kts`, Ela é usada no Android para ativar ou desativar o suporte para compilar arquivos AIDL (*Android Interface Definition Language*).

```
android {
    namespace = "com.leandromendes.aidl_interface"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.leandromendes.aidl_interface"
        minSdk = 26
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner =
            "androidx.test.runner.AndroidJUnitRunner"
    }
}
```

```

buildTypes {
    release {
        isMinifyEnabled = false
        proguardFiles(
            getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro"
        )
    }
}
compileOptions {
    sourceCompatibility = JavaVersion.VERSION_11
    targetCompatibility = JavaVersion.VERSION_11
}
// Enable build AIDL file
buildFeatures.aidl = true
}

```

2.3 Resultado

A imagem abaixo mostra o resultado da execução com os logs.

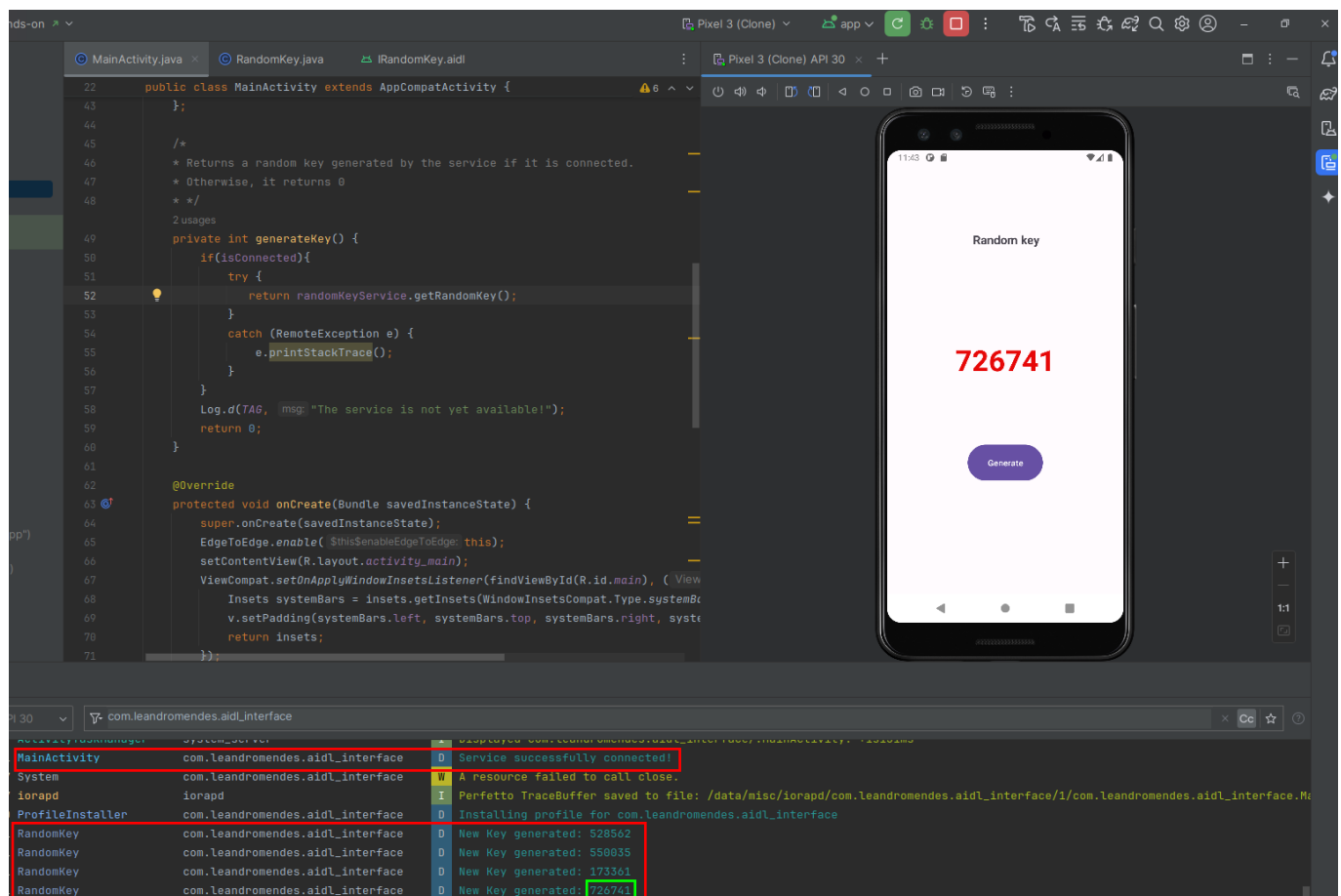


Figura 10: Resultado da atividade